Ai Agent 友好型 开发指南

这份指南的核心目标只有一个：

> **把软件工程，从“人写代码”，升级为“人设计系统 + AI执行 + 系统自动校验”。**

一、基础建设：默认架构

1. 推荐技术栈（默认解，后端可以根据特殊情况，调整）

代码块

```
Frontend:  Next.js (App Router)
Backend:   Python (FastAPI)
Repo:      Monorepo (JS pnpm + Py uv)
CI:        GitHub Actions
Deploy:    Vercel (frontend) + 自托管 (backend)
DB:        Postgres
Contract:  OpenAPI + Pydantic
```

---

2. 为什么必须 Monorepo（不是“建议”，是“前提”）

多 repo 的问题（AI时代更严重）

- 上下文割裂（frontend/backend/schema/test分散）
- AI 无法形成完整理解闭环
- 契约漂移（API / DB / UI 不一致）

Monorepo 的本质价值：

- **让“系统真相”集中在一个代码空间内**
- **Monorepo ≠ 管理方式**
- **Monorepo = All-in-one：代码 + 契约 + 测试 + 文档 + CI + Agent规则 + DB Migration，AI 选择上下文**

代码块

```
apps/
  web/
  api/
packages/
  schemas/
  sdk/
  ui/
openapi/
tests/
docs/
.github/
AGENTS.md
```

---

3. 为什么默认 Python + Next.js

Backend：FastAPI（强约束世界）

- 自动生成 OpenAPI（契约即代码）
- Pydantic → 强类型约束
- /docs + /openapi.json → agent 可读
- **Python + FastAPI：Ai 语料最丰富的语言 + 框架（可能没有之一）**

Frontend：Next.js（Agent 友好框架）

- CD 直接在 Vercel 上面做，方便
- App Router（结构清晰）
- Server Actions（简化数据流）
- 官方支持 AGENTS.md（给 AI 提供正确文档入口）

---

4. 为什么必须 Pre-commit + 强 CI 检查?

AI 的问题不是不会写代码，而是会“快速写错很多代码”

没有 CI 的后果

- AI commit 不可控
- 错误滞后爆炸
- 无法 scale agent

Pre-commit + Github Actions 协同

- 质量门禁：
  - Linter
  - unit tests
  - integration tests (smoke)
  - OpenAPI consistency
  - typecheck
  - etc...
- 不会写 Pre-commit + Github Actions 代码？ AI 可以自动生成对应编排代码；

二、Agent 时代的开发方法论（Harness Engineering）

1. 什么是 Harness Engineering

> **不是写 prompt，而是构建 AI 可稳定工作的环境**

你做的不是：

- 写代码

你做的是：

- 定义约束
- 设计结构
- 建立验证系统

---

2. 需求分级（非常关键）

A. 架构级（重度参与）

> 牵一发动全身

例子：

- 权限模型

- 异步架构
- 多租户
- 重度性能需求

让 AI 组合最佳实践，给你提方案，人做决策，进行契约制定

B. 对状态模型 / DB有修改（中度参与）

例子：

- 新表
- 状态机
- migration

让 AI 提方案，人做深度 Review，修改 Plan，试运行

C. 业务逻辑（轻度参与）

👉 AI 最强区

前提：

- 有测试
- 有结构
- 有契约

让 AI 完整实现，人对一些corner case进行 测试驱动

D. UI / 接口胶水（全自动）

👉 可以放心交给 AI

---

3. 核心原则（你刚刚强调的）

原则 1：不要在单点上挑战 AI

> **不要把质量控制建立在 AI 的单次 commit review 上**

❌ 错误方式

- review 每一个 commit
- 人肉找 bug

- 靠经验判断对错

✅ 正确方式

你要 review 的是：

- 需求契约
- 状态模型
- API 契约
- 测试覆盖
- 系统行为

一句话总结：

> **不要 review commit，要 review contract + system behavior**

原则 2：不要在跨模块任务上完全放手 AI

原因：

- 上下文爆炸
- 隐性依赖
- 错误不可见
- 正确方式：大任务 → 拆成可验证小闭环（这一步也可以让 Ai 代劳！根据需求复杂程度判断）

原则 3：多让AI 问你要决策：

每次复杂需求，使用 Plan Mode，告诉AI：有我说的模糊的地方，询问我；

三、如何解决 AI 上下文遗忘（核心能力）

1. 复杂需求：至少需要 3 个“彼此制约的角色”：规划、执行、评估

复杂需求，不要让一个 AI 同时做三件事：

- 想清楚要做什么
- 写代码实现
- 判断自己对不对

这是最容易失控的组合。

必须拆成三个角色：

规划 → 执行 → 评估

规划（Planner）

只做一件事：

> 把模糊需求，变成清晰约束

包括：

- 做什么 / 不做什么
- 成功标准是什么
- 拆成哪些阶段

不写代码。

执行（Generator）

只做一件事：

> 在约束内实现

不能：

- 改需求
- 改边界
- 自己决定完成标准

评估（Evaluator）

只做一件事：

> 判断是否真的完成

检查：

- 是否符合原始目标
- 是否满足约束
- 是否存在假实现 / stub
- 是否真的可用

为什么必须拆这三层

因为 AI 会天然做三件危险的事：

1. 一边写代码，一边改需求
2. 用“看起来差不多”替代“真的完成”
3. 给自己的结果打高分

如果不拆角色，这三件事会同时发生。

本质

不是要 3 个 agent，而是：

目标 ≠ 实现 ≠ 评估

这三件事必须隔离。

什么时候必须用三角色

满足任意一条就该拆：

- 涉及多个模块
- 涉及 DB / 状态
- 涉及 API 变更
- 执行时间长
- 需要测试验证
- 容易出现“看起来完成但其实没完成”

最小结构

Planner → 定义 contract

Generator → 实现

Evaluator → 验证

当任务边界清晰、影响范围局部、无需额外规划时，可以退化为“执行 + 评估”两角色结构：

2. 单点需求：执行、评估 彼此制约；

```
代码块
Session A → 执行
Session B → 评估（全新上下文）
```

- 长对话 = context 污染
- AI 会“忘早期约束”

---

3. 少用自然语言约束 AI（非常重要）

❌ 错误方式

- “这个接口要安全一点”
- “这个逻辑要严谨”
- “界面整洁一点”

✅ 正确方式

用结构：

- 用户同时发起两笔请求，不能出现双花；
- 用户未登录的情况下，不能访问该接口；
- 你的这次migration，是否破坏了任何业务层数据结构？

---

4. 必须沉淀三类“硬契约文件”

用户流程

```
代码块
draft → pending → paid → fulfilled
failure paths:
- payment failed
- inventory rollback
```

DB 状态模型

```
代码块
invariants:
- paid => paid_at != null
- cancelled != paid
```

测试规范

```
case: duplicate webhook
expect:
- idempotent
- no double side effects
```

5. 先想测试，再让 AI 干活

> 没有测试，AI 就没有收敛目标；
>
> 没有契约，测试也无法成立。

6. 用“短回路验证”替代“长回路相信”

```
每一步都必须能验证：
- lint
- test
- build
- contract check
```

四、Skills / CLAUDE.md / Subagents / AGENTS.md

这样理解：

- AGENTS.md：系统级规则
- CLAUDE.md：项目特殊补丁
- subagents：任务拆分执行
- skills：可复用流程

1. AGENTS.md（必须有）

👉 放“最小真相”

代码块

```
# Source of truth
- openapi.yaml
- docs/contracts/*
- migrations

# Rules
- no cross-layer import
- schema-first

# Required checks
- lint
- test
- openapi check
```

---

2. CLAUDE.md（极简）

👉 只写 AI 会忘的

代码块

```
- never edit generated SDK
- always add idempotency test for payments
- migration must include rollback
```

---

3. Subagents

参考：为什么我们需要规划、执行、评估

推荐拆：

代码块

```
root → orchestration
coding → implementation
testing → tests
review → diff check
```

```
docs → contract
```

👉 核心价值：

隔离上下文污染

分离职责

限制上下文范围

---

4. Skills（跨项目复用）

Skills 用来沉淀跨项目可复用的流程。

这一部分不预设清单，我们会在团队真实开发过程中逐步抽象和共享。